Root-Locus Lag Compensation

Improving steady-state accuracy without disturbing the transient.

Ogata, *Modern Control Engineering*, Ch. 6.

In this tutorial you will:

- size a lag compensator from a velocity-error-constant (K_v) spec,
- place its pole/zero pair near the origin to preserve the dominant poles, and
- confirm the transient is unchanged while ramp tracking improves.

Step through with **Ctrl+Enter**, or render a report with `publish`.

A phase-lag compensator

$G_c(s) = K_c \frac{s+z_c}{s+p_c}, \quad |z_c| > |p_c|$

(zero farther from the origin than the pole, both close together and close to the origin) is used to improve **steady-state error** without significantly disturbing the transient response/dominant closed-loop poles already achieving a satisfactory ζ, ω_n – the opposite design goal from lead compensation.

Contents

- Uncompensated Plant
- Static Velocity Error Constant
- Before: The Uncompensated Ramp-Tracking Error
- Design Goal
- Verifying the Dominant Poles Are Largely Unchanged
- After: What Changed – Ramp Tracking Improves
- What Did NOT Change: The Transient Response
- Where the Dominant Poles Moved (Hardly at All)
- Try it yourself

Uncompensated Plant

$G(s) = \frac{1}{s(s+1)(s+2)}$, already exhibiting acceptable transient response – its dominant closed-loop poles sit at $-0.34 \pm 0.56j$ ($\zeta \approx 0.5$) and we do not want to move them – but with insufficient velocity-error constant K_v.

```
G = tf(1,conv([1 0],conv([1 1],[1 2])));
```

Static Velocity Error Constant

For a type-1 system, $K_v = \lim_{s\to 0} sG(s)$, and the steady-state error to a unit ramp is $e_{ss} = 1/K_v$.

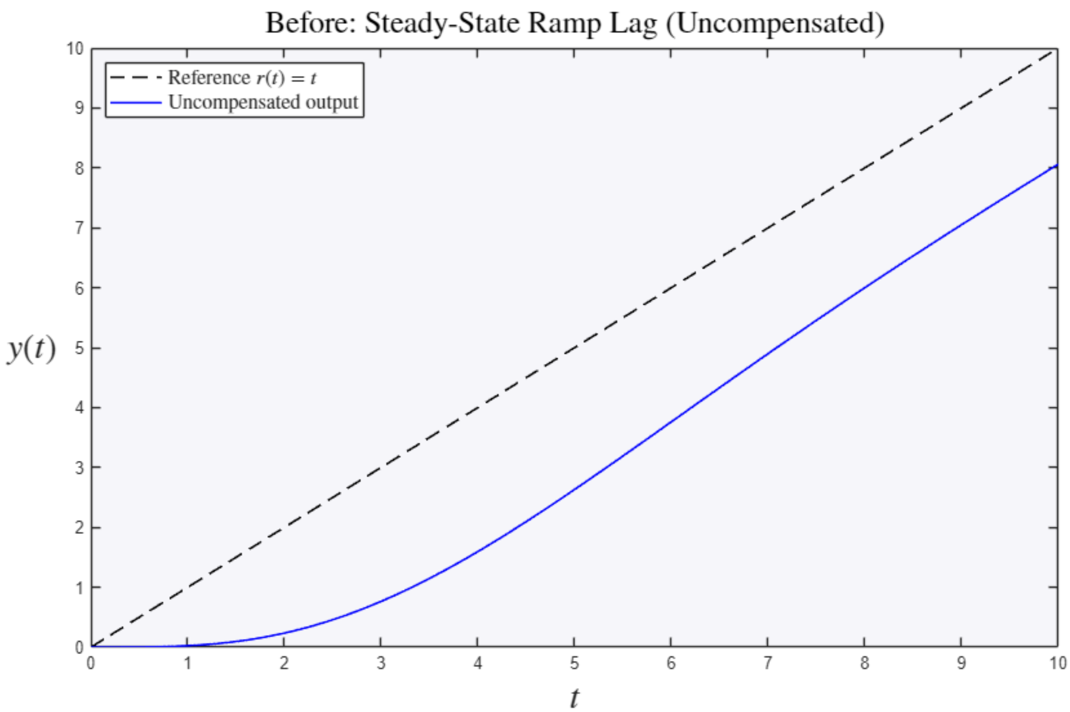
```
s = tf('s');
Kv_uncomp = dcgain(s*G);
fprintf('Uncompensated Kv = %.4f, ess(ramp) = %.4f\n', Kv_uncomp, 1/Kv_uncomp)
```

```
Uncompensated Kv = 0.5000, ess(ramp) = 2.0000
```

Before: The Uncompensated Ramp-Tracking Error

Lag compensation targets steady-state accuracy, so the telling "before" picture is the response to a unit ramp: the uncompensated output lags the reference by a constant ess = 1/Kv.

```
t_r = 0:0.01:10;
y_uncomp_ramp = lsim(feedback(G,1),t_r,t_r);
figure
plot(t_r,t_r,'k--',t_r,y_uncomp_ramp,'b','LineWidth',1.2)
legend('Reference $r(t)=t$','Uncompensated output','Interpreter','latex','FontSize',12,'Location','northwest')
title('Before: Steady-State Ramp Lag (Uncompensated)','Interpreter','latex','FontSize',18)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```



Design Goal

Suppose the specification requires $K_v \geq 5$ (i.e. $e_{ss} \leq 0.2$) while leaving the dominant closed-loop poles essentially unchanged. The lag compensator increases the gain "seen" at DC by the ratio z_c/p_c without changing the angle contribution near the dominant poles (since both z_c, p_c are placed close to the origin, far from the dominant poles, their net angle contribution there is small).

```
Kv_desired = 5;
ratio = Kv_desired/Kv_uncomp;
fprintf('Required zc/pc ratio = %.4f\n', ratio)

% Place pc very close to the origin (small, so both pole and zero stay
% far from the dominant poles and barely disturb the locus there) and
% compute zc from the ratio.
pc = 0.005;
zc = ratio*pc;
fprintf('Lag compensator pole at s = %.4f\n', -pc)
fprintf('Lag compensator zero at s = %.4f\n', -zc)

Gc = tf([1 zc],[1 pc]);
G_comp = series(Gc,G);

Kv_comp = dcgain(s*G_comp);
fprintf('Compensated Kv = %.4f, ess(ramp) = %.4f\n', Kv_comp, 1/Kv_comp)
```

```
Required zc/pc ratio = 10.0000
Lag compensator pole at s = -0.0050
Lag compensator zero at s = -0.0500
Compensated Kv = 5.0000, ess(ramp) = 0.2000
```

Verifying the Dominant Poles Are Largely Unchanged

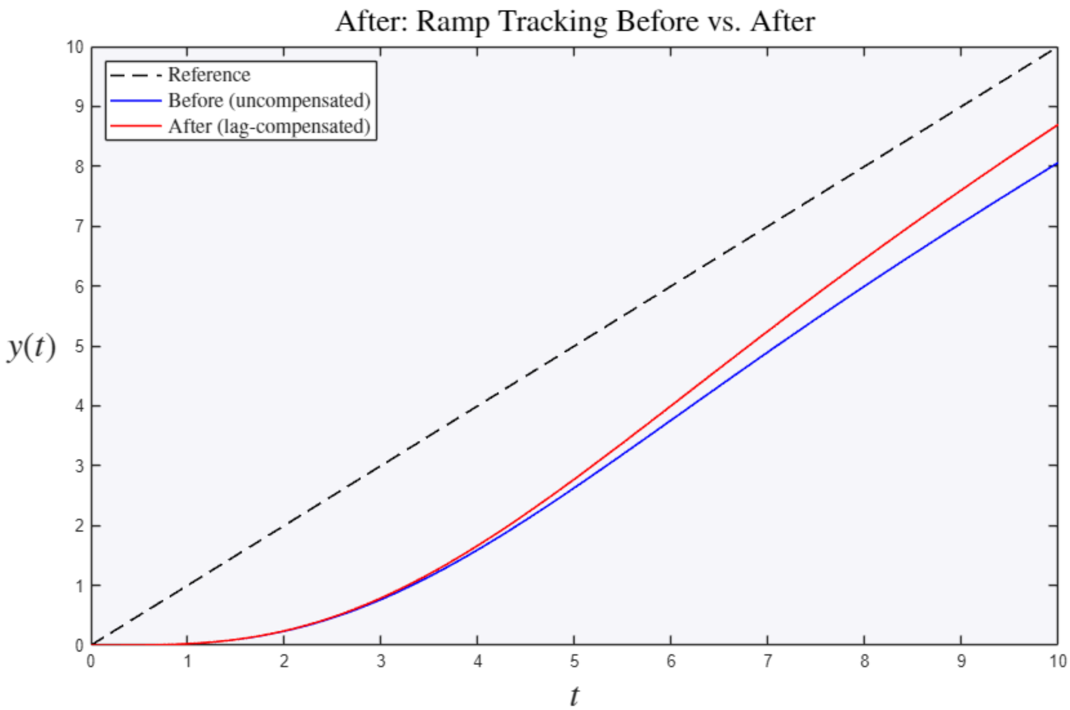
```
T_uncomp = feedback(G,1);
T_comp = feedback(G_comp,1);

poles_uncomp = pole(T_uncomp)
poles_comp = pole(T_comp)
```

```
poles_uncomp =

  -2.3247 + 0.0000i
  -0.3376 + 0.5623i
  -0.3376 - 0.5623i


poles_comp =

  -2.3201 + 0.0000i
  -0.3149 + 0.5405i
  -0.3149 - 0.5405i
  -0.0551 + 0.0000i
```

After: What Changed – Ramp Tracking Improves

The same ramp input, now with the compensator: raising Kv by the factor zc/pc shrinks the steady-state lag dramatically.

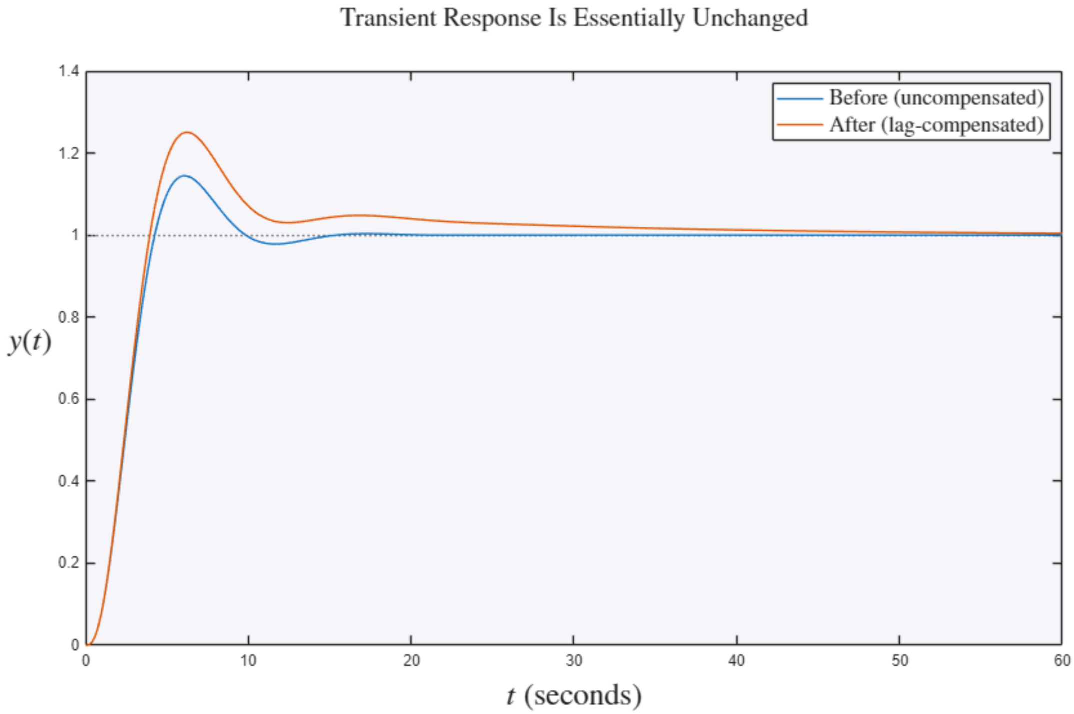
```
y_comp_ramp = lsim(T_comp,t_r,t_r);
figure
plot(t_r,t_r,'k--',t_r,y_uncomp_ramp,'b',t_r,y_comp_ramp,'r','LineWidth',1.2)
legend('Reference','Before (uncompensated)','After (lag-compensated)', ...
    'Interpreter','latex','FontSize',12,'Location','northwest')
title('After: Ramp Tracking Before vs. After','Interpreter','latex','FontSize',18)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```



What Did NOT Change: The Transient Response

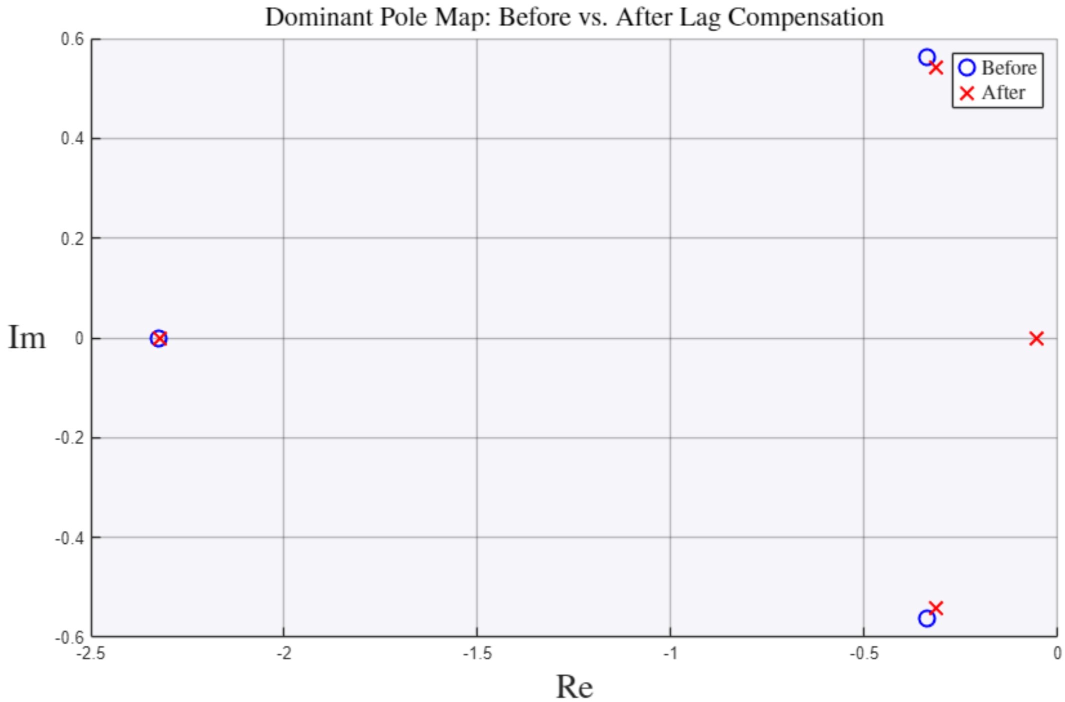
By design the dominant poles barely move, so the step (transient) response is almost identical – the whole point of lag compensation is to fix steady-state accuracy without disturbing the transient.

```
figure
hold on
step(T_uncomp)
step(T_comp)
hold off
legend('Before (uncompensated)','After (lag-compensated)','Interpreter','latex','FontSize',14)
title('Transient Response Is Essentially Unchanged','Interpreter','latex','FontSize',17)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```



Where the Dominant Poles Moved (Hardly at All)

```
figure
hold on
plot(real(poles_uncomp),imag(poles_uncomp),'bo','MarkerSize',9,'LineWidth',1.5)
plot(real(poles_comp),imag(poles_comp),'rx','MarkerSize',11,'LineWidth',1.5)
hold off
grid on
legend('Before','After','Interpreter','latex','FontSize',12)
title('Dominant Pole Map: Before vs. After Lag Compensation','Interpreter','latex','FontSize',16)
ylabel('$\mathrm{Im}$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$\mathrm{Re}$','Interpreter','latex','FontSize',20)
```



Summary: Kv (and ramp accuracy) improves by the factor zc/pc while the dominant closed-loop poles – and hence the transient response – are left essentially unchanged.

Try it yourself

- Demand `Kv_desired = 20` instead of 5 and notice the zero/pole ratio (and the ramp accuracy) increase, with the transient still nearly untouched.
- Move the pole out to `pc = 0.2` and watch it disturb the dominant poles – in fact it pushes them into the right half-plane (unstable): a lag pole/zero pair must stay near the origin.